

Bases de Données Relationnelles

Licence 2 MPI / SML



Langage SQL

Pr Amadou Dahirou GUEYE, enseignant-chercheur à l'UFR STA, UAM

Chapitre 2 : Langage SQL

Objectifs spécifiques : A la suite de ce chapitre, l'étudiant doit être capable de:

1. Comprendre le langage SQL
2. Identifier les différents sous langages de SQL
3. Créer la structure d'une table
4. Modifier la structure d'une table
5. Supprimer une table
6. Définir les différentes contraintes d'intégrités
7. Créer les contraintes d'intégrités
8. Modifier les contraintes d'intégrités
9. Définir une requête
10. Effectuer des requêtes simples ou complexes
11. Ajouter des colonnes

Le langage SQL

Qu'est-ce que le langage SQL ?

- Le langage SQL (Structured Query Language) est un langage d'interrogation d'une base de données.
- Il permet la définition, la manipulation et le contrôle des bases de données relationnelles.
- Il est composé de cinq sous-langages :
 - Langage de Définition de Données (LDD) : création, modification et suppression des définitions des tables.
 - Langage de Manipulation de Données (LMD) : ajout, suppression, modification et interrogation des données.
 - Langage d'Interrogation de Données (LID) : L'exploitation des données.
 - Langage de Contrôle de Données (LCD) : gestion des accès multiutilisateur aux données. (Traité dans le chapitre suivant)
 - Langage de Contrôle des Transactions (LCT) : gestion des transactions (validation et annulation des transactions). (Traité dans le chapitre suivant)

Le langage SQL

Langage de Définition de Données (LDD)

- Le langage de définition de données permet de définir les objets de la base de données.
- Il repose essentiellement sur trois mots clés :

➤ CREATE pour créer les objets ;

➤ DROP pour supprimer les objets ;

➤ ALTER pour modifier les objets .

Le langage SQL

Les tables

- Création

```
CREATE TABLE nom_table (  
    attribut1 type1,  
    attribut1 type1,  
    attribut1 type1,  
    Contrainte nom_contrainte_primaire,  
    Contrainte nom_contrainte_etrangere,  
    Contrainte nom_contrainte_domaine  
);
```

Le langage SQL

Les tables

- Création

Chaque attribut (ou colonne) appartient à un type et on en rencontre plusieurs parmi lesquels:

Types de données	Description
INT(n)	Entier à n chiffres
NUMERIC(n, m)	Réel à n chiffres. m est le nombre de chiffres après la virgule
VARCHAR(n)	Chaîne de n caractères
DATE	JJ-MM-AAAA ou AAAA-MM-JJ
BLOB	Les objets binaires de grande taille (son, vidéo, image)

Le langage SQL

Les contraintes d'intégrité

- **Clé primaire:** est l'attribut permet d'identifier chaque enregistrement d'une table de manière unique.

Syntaxe : CONSTRAINT nom_contrainte PRIMARY KEY
(attribut_clé1, attribut_clé2,....)

- **Clé étrangère :** est une clé primaire provenant d'une autre table.

Syntaxe : CONSTRAINT nom_contrainte FOREIGN KEY
(attribut_clé) REFERENCES table(attribut)

- **Contrainte de domaine :** est une contrainte de recherche

Syntaxe : CONSTRAINT nom_contrainte CHECK (condition)

Le langage SQL

Exemple

```
CREATE TABLE Client (  
    numClient int,  
    nom varchar(15),  
    prenom varchar(20),  
    adresse varchar(30),  
    Constraint Client_pk PRIMARY KEY(numClient),  
);  
:
```


Le langage SQL

Les tables

- **Modifications de la structure d'une table**

Elle peut se faire comme suit:

- **Par l'ajout d'un attribut**

ALTER TABLE nom_table ADD (lattribut_concerné TYPE);

Exemple : ALTER TABLE Client ADD (email varchar(30));

- **Par modification du nom ou type d'un attribut**

ALTER TABLE nom_table MODIFY(attribut_concerné TYPE);

Exemple : ALTER TABLE Client MODIFY (adresse varchar(40));

- **Par suppression d'un attribut**

ALTER TABLE nom_table DROP COLUMN attribut_concerné;

Exemple : ALTER TABLE Client DROP COLUMN adresse;

Le langage SQL

Les tables

- **Modifications de la structure d'une table**

Elle peut se faire comme suit:

- **Par l'ajout d'une contrainte**

```
ALTER TABLE nom_table ADD CONSTRAINT nom_contrainte
```

```
TYPE_de_contrainte(nom_attribut);
```

Exemple : ALTER TABLE Client ADD CONSTRAINT Client_pk PRIMARY KEY(numClient);

- **Par suppression d'une contrainte**

```
ALTER TABLE nom_table DROP CONSTRAINT nom _contrainte;
```

Exemple : ALTER TABLE Client DROP CONSTRAINT numClient;

Le langage SQL

Les tables

- **Copie et suppression (ou destruction) d'une table**

- Copie d'une table

CREATE TABLE nom_table_copie AS requête;

Exemple : CREATE TABLE ClientBIS AS SELECT * FROM Client;

- Suppression d'une table

DROP TABLE nom_table ;

Exemple : DROP TABLE Client;

Le langage SQL

Langage de Manipulation de Données (LMD)

- Le langage de manipulation de données permet de manipuler les données de la base de données. Il repose essentiellement sur trois mots clés :
 - INSERT pour insérer les données ;
 - UPDATE pour mettre à jour les données ;
 - DELETE pour supprimer les données ;

Langage d'Interrogation de Données (LID)

- Le langage d'interrogation de données permet d'interroger les données à l'aide du mot clés:
 - SELECT pour interroger les données.

Le langage SQL

Insertion, mis à jour et suppression de données

- **Insertion de données dans une table : enregistrement**

```
INSERT INTO nom_table( attribut1, attribut2,.....,attributN) VALUES(Valeur_attribut1,  
Valeur_attribut2, ...,valeur_attributN);
```

ou bien

```
INSERT INTO nom_table VALUES(Valeur_attribut1, Valeur_attribut2, ..., valeur_attributN);
```

Exemple : INSERT INTO Client VALUES(1, 'Diop', 'Awa', 'Pikine');

- **Mise à jour de la valeur d'un attribut**

```
UPDATE nom_table SET attribut=nouvelle_valeur [WHERE condition];
```

Exemple : UPDATE Client SET adresse='Ouakam' WHERE numClient=1;

- **Suppression d'un enregistrement**

```
DELETE FROM nom_table [WHERE condtion];
```

Exemple : DELETE FROM Client WHERE numClient =1;

Le langage SQL

Interrogation des données

- **Afficher de tous les n-uples d'une table**

SELECT * FROM nom_table;

Exemple1 : SELECT * FROM Commune;

Exemple2 : SELECT * FROM Client;

- **Forme générale (Tri et Agrégation)**

SELECT [ALL | DISTINCT] liste d'attributs de partitionnement à projeter et de fonction de calcul

FROM liste de relations

[WHERE condition à appliquer avant calcul de l'agrégat]

[GROUP BY liste ordonnée d'attributs de partitionnement [HAVING condition2]

[ORDER BY liste_attributs3 [ASC | DESC]];

Le langage SQL

Interrogation des données: Exemple 1: Tri

Il est possible de classer le résultat d'une requête en ajoutant le mot clef **ORDER BY** suivi d'une liste de champs.

Exemple : **SELECT * FROM** commune **ORDER BY** nom_comm
pour classer le résultat par nom de commune.

Un tri décroissant peut-être obtenu en ajoutant **DESC**.

Exemple : **SELECT * FROM** commune **ORDER BY** nom_comm **DESC**

SELECT nom_comm, **round**(cast(population as float)/superficie,2) **AS** densite
FROM commune **ORDER BY** densite
retourne la densité de population par ordre croissant de densité.

Résultat :

	NOM_COMM	densite
1	VAULANDRY	0.11
2	COURCELLES-LA-FORET	0.2
3	THOREE-LES-PINS	0.25
4	SAINT-JEAN-DE-LA-MOTTE	0.28
5	BOUSSE	0.33
6	CLEFS	0.35

Le langage SQL

Interrogation des données: Exemple 2: Agrégation

Une agrégation est une opération qui permet de regrouper les enregistrements de la table en sortie selon des critères et d'obtenir des informations statistiques sur ces regroupements. Il faut utiliser l'expression **GROUP BY** suivi du critère de regroupement.

Prenons un exemple à partir de la table COMMUNE. Nous souhaitons obtenir la population totale par département.

SELECT Nom_comm, nom_dept, population **FROM** commune
nous donne :

Le langage SQL

Interrogation des données: Exemple 2: Agrégation

	NOM_COMM	NOM_DEPT	POPULATION
1	SAINT-JEAN-DE-LA-MOTTE	SARTHE	900
2	ARTHEZE	SARTHE	400
3	VAULANDRY	MAINE-ET-LOIRE	300
4	CLEFS	MAINE-ET-LOIRE	900
5	MAREIL-SUR-LOIR	SARTHE	600
6	BOUSSE	SARTHE	400
7	LE BAILLEUL	SARTHE	1200
8	CLERMONT-CREANS	SARTHE	1200
9	MALICORNE-SUR-SARTHE	SARTHE	2000
10	THOREE-LES-PINS	SARTHE	700
11	LA FONTAINE-SAINT-MARTIN	SARTHE	600
12	LA FLECHE	SARTHE	15400
13	VILLAINES-SOUS-MALICORNE	SARTHE	1000
14	CRE	SARTHE	800
15	CROSMIERES	SARTHE	900
16	SAINT-QUENTIN-LES-BEAUREPAIRE	MAINE-ET-LOIRE	300
17	BAZOUGES-SUR-LE-LOIR	SARTHE	1200
18	COURCELLES-LA-FORET	SARTHE	400
19	LIGRON	SARTHE	500

Population des communes

Le langage SQL

Interrogation des données: Exemple 2: Agrégation (suite)

la requête :

```
SELECT nom_dept, sum(population) AS population_dept FROM commune  
GROUP BY nom_dept
```

renvoie :

	NOM_DEPT	population_dept
1	MAINE-ET-LOIRE	1500
2	SARTHE	28200

Agrégation par département

Le langage SQL

La clause GROUP BY fonctionne de concert avec les fonctions d'agrégation (ici sum()).
Les principales fonctions d'agrégation sont :

count() : renvoie le nombre d'enregistrements

sum() : renvoie la somme

max() : maximum

min() : minimum

avg() : moyenne

Le langage SQL

Interrogation des données: Exemple 2: Agrégation (suite)

Clause Having

Il se peut que l'on souhaite mettre un critère de sélection sur une colonne calculée par l'opération d'agrégation. Dans l'exemple ça serait le cas si on souhaite n'afficher que les départements de plus de 20000 habitants. On pourrait être tenté d'écrire une requête de la forme :

```
SELECT nom_dept, sum(population) AS population_dept FROM commune  
WHERE population_dept > 20000 GROUP BY nom_dept
```

ça ne marche pas car la clause where est exécutée avant l'agrégation. La clause **HAVING** permet d'indiquer au SQL d'effectuer une nouvelle sélection à la fin du calcul sur les résultats du regroupement. On écrira donc:

```
SELECT nom_dept, sum(population) AS population_dept FROM commune  
GROUP BY nom_dept HAVING population_dept > 20000
```

Le langage SQL

Interrogation des données

- Exemple 3

SELECT [ALL | **DISTINCT**] soit la table « vacataires », on veut afficher le nom des vacataires sans Redondances.

```
SELECT DISTINCT Nom FROM  
vacataires;
```

Résultat :

OMAR
SOULEY
MOUHAMED

Nom	Heure_vacation	JOUR
OMAR	12	1
SOULEY	11	1
MOUHAMED	14	1
OMAR	13	2
SOULEY	12	2
MOUHAMED	15	2
OMAR	19	3

Le langage SQL

Interrogation des données

- Exemple 3

[**GROUP BY**] : soit la table « vacataires », on veut connaître le nombre d'heures de vacances effectuées pour chaque vacataire:

```
SELECT Nom, SUM(Heure_vacation)
FROM vacataires GROUP BY Nom ;
```

Résultat :

OMAR = 44

SOULEY = 23

MOUHAMED = 29

Nom	Heure_vacation	JOUR
OMAR	12	1
SOULEY	11	1
MOUHAMED	14	1
OMAR	13	2
SOULEY	12	2
MOUHAMED	15	2
OMAR	19	3

Le langage SQL

Interrogation des données

- Exemple 3 :

[**GROUP BY**] [**HAVING** condition2]

Soit la table « vacataires », on souhaite déterminer et afficher le nombre d'heures de vacation effectuées par chaque vacataire et dont le total dépasse 35 heures.

```
SELECT nom, SUM(Heure_vacation)
FROM vacataires GROUP BY nom
HAVING SUM(Heure_vacation) > 35 ;
```

Résultat :

OMAR = 44

MOUHAMED = 39

nom	Heure_vacation	JOUR
OMAR	12	1
SOULEY	11	1
MOUHAMED	24	1
OMAR	13	2
SOULEY	12	2
MOUHAMED	15	2
OMAR	19	3

Le langage SQL

Interrogation des données

- Exemple 3

[**ORDER BY**] liste_attributs2 soit la table personne, on veut avoir une liste de nom et de prénom et nous voulons que cela soit ordonné par l'Age et par ordre croissant.

```
SELECT nom, prenom FROM personne  
ORDER BY age;
```

Résultat :

```
ANNE SADA  
NIANE CHEIKH  
DIAW OMAR
```

age	nom	prenom
39	DIAW	OMAR
27	ANNE	SADA
32	NIANE	CHEIKH

Le langage SQL

Interrogation des données

- Exemple 3

[**ORDER BY**] : soit la table personne, on souhaite avoir une liste de nom et de prénom et nous voulons que cela soit ordonné par l'Age et par ordre **décroissant**.

```
SELECT nom, prenom FROM personne  
ORDER BY age DESC;
```

Résultat :

```
DIAW OMAR  
NIANE CHEIKH  
ANNE SADA
```

age	nom	prenom
39	DIAW	OMAR
27	ANNE	SADA
32	NIANE	CHEIKH

Le langage SQL

Interrogation des données

- **Une projection**

SELECT liste_attributs1 FROM nom_table;

Exemple : SELECT nom, prenom FROM Client;

- **Une sélection**

SELECT * FROM nom_table WHERE condition;

Exemple : SELECT nom, prenom FROM Client WHERE adresse='Pikine';

- **Le renommage**

SELECT attribut AS nouveau_nom FROM nom_table;

Exemple : SELECT adresse AS 'habite à' FROM Client;

Le langage SQL

Interrogation des données

- **Union de requêtes**

```
SELECT liste_attributs1 FROM nom_table1
```

```
UNION
```

```
SELECT liste_attributs1 FROM nom_table2 ;
```

- **Intersection de requêtes**

```
SELECT * FROM nom_table1 INTERSECT SELECT * FROM nom_table2 ;
```

- **La différence**

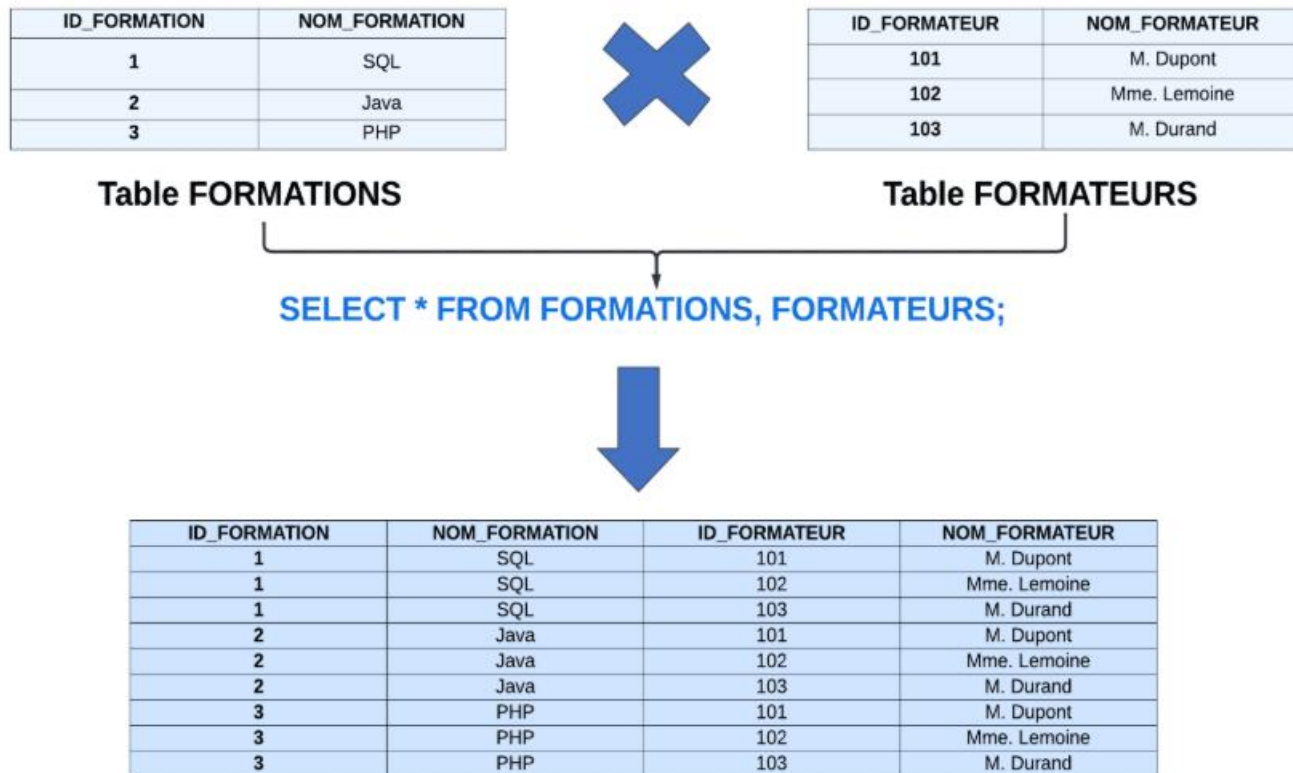
```
SELECT * FROM nom_table1 MINUS SELECT * FROM nom_table 2;
```

Le langage SQL

Interrogation des données

- **Produit cartésien** (une sorte de jointure sans condition)

SELECT liste_attributs FROM nom_table1, nom_table2 ;



Le langage SQL

Interrogation des données

- **La jointure**

SELECT liste_attributs FROM nom_table1 NATURAL JOIN nom_table 2;

Exemple : SELECT * FROM Client NATURAL JOIN Commandes;

- **La jointure externe**

SELECT liste_attributs FROM nom_table1 OUTER JOIN nom_table 2 ON
nom_table1.id_table1=nom_table2.id_table1_fk;

Exemple : SELECT * FROM Client OUTER JOIN Commandes ON
Client.numClient=Commandes.numClient;

Le langage SQL

Interrogation des données

- **La jointure externe gauche**

```
SELECT liste_attributs FROM nom_table1 LEFT OUTER JOIN nom_table2 ON  
nom_table1.id_table1=nom_table2.id_table1_fk;
```

Exemple : SELECT * FROM Client LEFT OUTER JOIN Commandes
ON Client.numClient=Commandes.numClient;

- **La jointure externe droite**

```
SELECT liste_attributs FROM nom_table1 RIGHT OUTER JOIN nom_table2 ON  
nom_table1.id_table1=nom_table2.id_table1_fk;
```

Exemple : SELECT * FROM Client RIGHT OUTER JOIN Commandes
ON Client.numClient=Commandes.numClient;

Application

- Soit le schéma relationnel suivant :

Acheteur(idAcheteur, nom, prenom, tel, adresse email, #ninea)

Vendeur(numVendeur, nom, prenom, adresse, email, tel, #ninea)

Articles(numArt, designation, prix_unitaire, quantitéStock, #ninea)

Boutique(ninea, adresse, chiffreAffaire, tel,)

Travail à faire: Donner les commandes SQL correspondant aux requêtes suivantes:

1. La liste des vendeurs.
2. La liste des potentiels acheteurs
3. La liste de tous les articles d'une boutique
4. La liste de toutes les boutiques
5. Les articles qui ont été achetés
6. Les articles vendus
7. Les articles achetés et vendus par la même personne c'est-à-dire l'acheteur est le vendeur

Application

8. Les vendeurs qui vendent les mêmes articles
9. Les acheteurs habitant Pikine.
10. Les vendeurs et acheteurs ayant la même adresse
11. Les articles dont le stock est inférieur à 10.
12. Les articles achetés au vendeur Abdou Faye.
13. Les articles achetés à la boutique 18 et dont le prix_unitaire est égale à 15 000.
14. Le vendeur Doudou Gueye qui a vendu l'article puce orange 4g, à l'acheteur Alimatou BA
15. Les boutiques situées dans la même zone que ses acheteurs et vendeurs.
16. Les Vendeurs portant le Diop.
17. Les achats dont le montant est compris entre 150 000 et 250 000
18. Les articles dont le nom commence par C et O.
19. Les boutiques dont le chiffre d'affaires dépasse les 1 000 000
20. Les boutiques qui ont vendus beaucoup plus d'articles.